

UPCL: Reviewer’s Guide to the Code

Companion to “A Unified Probabilistic Context Layer for
Risk-Sensitive Decision-Making Under State Uncertainty” (anonymous submission)

1 In one paragraph

An agent rarely observes the state of its environment directly: it infers it from noisy sensors or predictive models. Most pipelines nevertheless collapse that inference to a single most-likely state *before* acting, which discards exactly the information a risk-sensitive decision rule needs. UPCL keeps the uncertainty. It represents the uncertain state as a probability distribution over *possible worlds*, scores every candidate action in every world, and aggregates the world-wise scores with a pluggable risk criterion (Expected Utility, Maxmin, Hurwicz, CVaR, DRO) before selecting an action. The belief is built independently of the policy that consumes it, so an existing recommender or controller can be made uncertainty-aware without being rewritten. This repository implements the framework, checks every formal statement of the paper with a unit test, and reproduces every reported number from a script and a frozen log.

2 The approach

1. Belief construction. A *probabilistic context instance* $\kappa_u^t = \{P(C_1), \dots, P(C_K)\}$ gives, for each discrete state variable C_k (mood, location, ...), a distribution over its values. The transformation Φ turns it into a probabilistic context space $(\Omega, \mathbb{P})$: Ω is the set of complete assignments (possible worlds) and $\mathbb{P}$ their probabilities, under tuple-independent (TID), block-independent-disjoint (BID) or hybrid semantics borrowed from probabilistic databases.

2. Aggregation and decision. For a user u and item (action) i , a policy produces a world-wise score $r(u, i, \omega)$. A generic operator Γ_θ maps $\{(r(u, i, \omega), \mathbb{P}(\omega))\}_\omega$ to one score $S_\theta(u, i)$; the decision operator $\Psi = \arg \max_i S_\theta(u, i)$ then picks the action. Changing the risk attitude changes only Γ_θ ; Φ and Ψ are untouched.

3. Three integration points. The same belief can meet the policy at three places, so one belief layer serves policies of different kinds:

- **UPCL-Pre:** evaluate the policy under every world, then aggregate (needs a policy callable per world).
- **UPCL-Model:** encode the belief into a fixed-size vector z and feed it to a learned policy $f_\varphi(u, i, z)$ (one call, needs training).
- **UPCL-Post:** call a black-box base policy $f_0(u, i)$ once, adjust its score per world with h_η , then aggregate (works for policies that cannot be retrained).

4. Scale. Enumerating Ω costs $\prod_k m_k$. A Monte Carlo estimator samples N worlds and carries a Hoeffding finite-sample bound, with a closed-form rule $N \geq \frac{(b-a)^2}{2\varepsilon^2} \ln \frac{2}{\delta}$ for a target accuracy.

A ten-second illustration. Alice’s mood is Happy with probability 0.65, Tired with 0.35. Item A scores 4.8/3.2 in the two worlds, item B scores 3.9/4.5. Expected Utility gives 4.24 for A and 4.11 for B, so A is chosen; Maxmin gives 3.2 and 3.9, so B is chosen. The belief, the world-wise scores and Ψ are identical; only Γ_θ changed. (`experiments/example_alice.py` reproduces this.)

3 What it adds relative to existing work

The paper positions UPCL against four bodies of work, none of which covers all of the following three requirements at once: (i) a declarative, policy-agnostic belief over the agent’s state; (ii) a decision layer linking that belief to actions under both risk-neutral and risk-sensitive criteria; (iii) independence from the policy that consumes the belief.

Line of work	What it provides	What is missing for the setting above
Belief-state planning (POMDPs)	A principled belief over hidden state	Belief and policy are coupled in one solver; no pluggable risk criterion over an <i>existing</i> policy
Risk-sensitive decision theory (Maxmin, Hurwicz, CVaR, DRO)	Criteria for acting under a distribution or ambiguity set	Assumes a belief is already available; says nothing about constructing it
Probabilistic databases (TID/BID, possible worlds)	Semantics for uncertain data	No decision layer; no notion of acting on the result
Context-aware recommenders (pre-/post-filtering, contextual models)	Use of situational context	Context treated as exact or absent; uncertainty collapsed before acting
Bayesian / conformal uncertainty for recommenders	Calibrated predictive uncertainty on scores	Uncertainty on the <i>prediction</i> , not on the observed <i>state</i> ; one point-estimate rule at a time

The contribution is therefore a *composition*: probabilistic-database semantics to represent the state, a generic risk operator to decide, and three integration points to attach the whole to any policy, together with (a) consistency and reduction guarantees (a degenerate belief recovers the deterministic decision exactly), (b) Monte Carlo convergence and finite-sample bounds, and (c) an impossibility result showing that handing the policy a single realisation cannot preserve uncertainty while remaining policy-independent. The evaluation uses context-aware recommendation because its ground truth is tractable; nothing in the framework is specific to it.

4 Code map

Module	Role (paper reference)
upcl/context.py	ContextualVariable, ProbabilisticContextInstance, ProbabilisticContextSpace; $\phi()$ implements Φ with TID/BID/hybrid semantics (Defs. 1–7)
upcl/aggregation.py	Γ_θ : expected_utility, maxmin, hurwicz, cvar, dro (Section 4.7)
upcl/decision.py	Ψ : psi, psi_ranked
upcl/instantiations.py	upcl_pre, upcl_model, upcl_post; same return type (best_item, {item: score}) (Section 5)
upcl/monte_carlo.py	Sampling estimator, hoeffding_bound, required_sample_size (Section 6.2)
upcl/noise.py	inject_noise (uniform-mix or Dirichlet) and collapse_to_mode, the deterministic baseline
upcl/stats.py	Paired t -test, Wilcoxon, paired Cohen’s d , Holm–Bonferroni
upcl/recommenders/toy.py	Seed-reproducible stand-in recommenders for the synthetic testbed
upcl/datasets/loaders.py	CSV loader for real context-aware benchmarks

Minimal usage.

```
from upcl.context import ContextualVariable, ProbabilisticContextInstance, phi
from upcl.instantiations import upcl_pre

mood = ContextualVariable("Mood", {"Happy": 0.65, "Tired": 0.35})
space = phi(ProbabilisticContextInstance(categorical_variables=[mood]))

# any callable f(user, item, world) -> score; policy="eu" | "maxmin" |
# "hurwicz" | "cvar" | "dro" (extra kwargs, e.g. alpha=, beta=, are forwarded)
best, scores = upcl_pre("Alice", ["A", "B"], space, my_policy, policy="maxmin")
```

Design choices worth knowing.

- One call signature for all three strategies and all five operators, so swapping a risk criterion or an integration point is a one-argument change.
- Worlds are tuples of (*variable*, *value*) pairs, so a policy can read only the variables it cares about; zero-probability worlds are pruned by default.
- UPCL-Post treats the base policy as a black box (`base_recommender(u,i)`); only the adjustment h_η and Γ_θ see the belief.
- Everything stochastic takes an explicit seed; the experiment scripts are deterministic, and the committed logs are their exact output.

5 Checking and reproducing the claims

Unit tests (one module per component; `pytest tests/ -v`, or `python run_tests.py` without `pytest`): normalisation of TID/BID/hybrid worlds, singleton consistency and reduction to the deterministic case, Hurwicz bounds, exact reproduction of the running example for UPCL-Pre and UPCL-Post, Monte Carlo convergence and seed reproducibility, and agreement of `required_sample_size` with the Hoeffding bound.

Experiments. Every script lives in `experiments/`; its output is frozen in `outputs/`, so a number can be checked without re-running anything.

Result in the paper	Script	Frozen output
Running example	<code>example_alice.py</code>	printed to console
RQ1–RQ3, RQ5–RQ6 (synthetic)	<code>full_evaluation.py</code>	<code>full_evaluation.log.txt</code>
Statistical-rigor table (RQ1/RQ2)	<code>rigor_and_baseline.py</code>	<code>rigor_and_baseline.log.txt</code>
UPCL-Model, synthetic structured testbed	<code>upcl_model_evaluation.py</code>	<code>upcl_model_evaluation.log.txt</code>
RQ4 risk-sensitive aggregation	<code>rq4_full_sweep.py</code>	<code>rq4_full_sweep.log.txt</code>
CVaR/DRO hyperparameter ablation	<code>rq4_hyperparam_sensitivity.py</code>	<code>rq4_hyperparam_sensitivity.log.txt</code>
DePaulMovie, LDOS-CoMoDa (UPCL-Post)	<code>real_dataset_multiseed.py</code>	<code>depaulmovie_*, ldos_comoda_*</code>
Frappe (UPCL-Post)	<code>real_dataset_multiseed_fast.py</code>	<code>frappe_multiseed.log.json</code>
Frappe (UPCL-Model)	<code>frappe_model_evaluation.py</code>	<code>frappe_model_evaluation.log.json</code>
MF capacity / shrinkage ablation	<code>mf_shrink_ablation.py</code>	<code>mf_shrink_k_ablation.json</code>

The synthetic scripts need no data. The real-data scripts need the raw benchmark from its official source (no dataset is redistributed); `prepare_real_dataset.py` converts it to the tidy schema, and `README.md` lists the exact commands.

Evaluation protocol. Robustness is measured by moving each state variable toward the uniform distribution with intensity $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ and comparing against a baseline that collapses the (noisy) belief to its mode. Every comparison is a paired two-sided t -test with paired Cohen’s d ; where a claim rests on several tests, Holm–Bonferroni correction is applied across the whole family.

6 Summary of results

Setting	Outcome
Synthetic, decision quality ($\lambda = 0.3$, 100 agents)	NDCG@5: non-contextual 0.774, deterministic 0.940, UPCL-Pre 0.997; the gains are significant after Holm–Bonferroni correction (Cohen’s $d = 2.40$ and 1.33)
Synthetic, robustness	UPCL-Pre beats the deterministic baseline at every noise level; the effect size shrinks from $d = 1.44$ ($\lambda = 0$) to $d = 0.34$ ($\lambda = 1$) as both approach an uninformative belief
Synthetic, risk–return (RQ4)	Expected Utility has the best mean and the worst 5th-percentile outcome; Maxmin, CVaR and DRO trade mean for a better worst case; the trade-off is continuous in the Hurwicz coefficient. Against a Bayesian-LCB baseline the picture is mixed: parity, not dominance
Monte Carlo	Empirical error stays below the Hoeffding-derived target; exact enumeration grows with $ \Omega $ while sampling grows far more slowly
Real data, UPCL-Post	Frappe: significant gain at every noise level above 0. DePaulMovie and LDOS-CoMoDa (far sparser per item–context cell): same direction, not significant after correction
UPCL-Model	Beats the non-contextual and deterministic baselines on the synthetic structured testbed but remains below the oracle-based UPCL-Pre/Post; on Frappe it loses to the non-contextual backbone, which we report as an open limitation

7 Scope and anonymity

The framework assumes discrete state variables and, for the closed-form TID/BID constructions, independence between blocks (a joint model can replace the probability-assignment step). The repository contains no author names, affiliations or identifying history, and it redistributes no third-party dataset; it is meant to be browsed through an anonymised mirror during review.